

SIMATSENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO4: Develop intelligent software agents using suitable agent architectures and learning techniques. (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

I Kamalli shree V(192525030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	

9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11.LimitationsandFuture Enhancements	5%	
12.DocumentationandTechnicalPresentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario/Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	Smart Loan Approval: A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	Student Performance Prediction: A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	Disease Risk Prediction: A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.

4	Image Recognition System: An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	Handwritten Character Recognition: A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.
6	Non-Linear Classification: A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	Intelligent Recommendation Agent: An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	Autonomous Game Agent: An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.

9	Explainable Diagnosis System: A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	AI Model Selection Challenge: A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	Problem Statement	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	Objectives	State the specific goals of developing the AI model and expected outcomes.
3	Dataset Description	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	Data Preprocessing	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	Selected AI Technique and Justification	Identify the selected AI technique and justify its suitability for the problem.
6	Model Design/Architecture	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	Algorithm/Pseudocode	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	Implementation	Provide programming language, libraries/tools, important code sections, and implementation details.
9	Experimental Results	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	Performance Evaluation	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	Result Analysis	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	Limitations	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	Future Enhancements	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	Conclusion	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.

15	References	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	GitHub Repository Link	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

1. Problem Statement

An organization wants to automatically classify images into three categories: Animals, Vehicles, and Buildings. Manual classification of a large number of images can be time-consuming and may lead to inconsistent results.

The proposed system uses a Convolutional Neural Network (CNN) to learn visual features from labeled images and classify an input image into one of the three predefined categories.

Input

- RGB image of size $32 \times 32 \times 3$

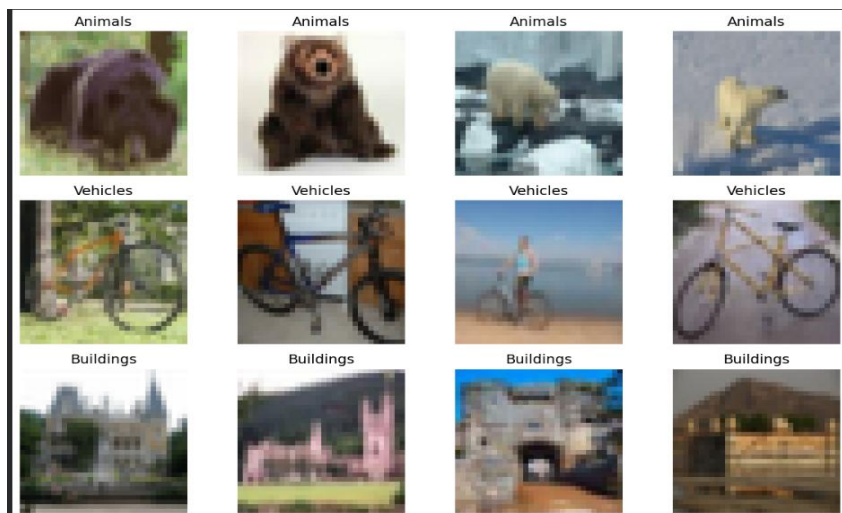


Fig.1 Sample input images used for classification into

Animals, Vehicles, and Buildings.

AI Task

- Multi-class image classification
- Classes: Animals, Vehicles, Buildings

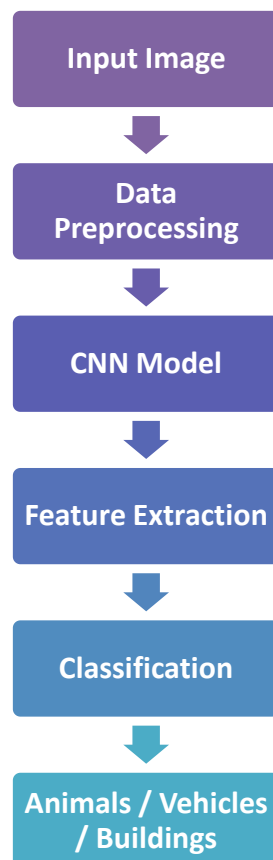
Expected Output

The trained neural network predicts the category of the given image.

Problem Objective

To design and implement a neural-network-based image recognition system that can automatically classify images into Animals, Vehicles, or Buildings and evaluate its performance using accuracy, precision, recall, F1-score, and a confusion matrix.

System Flow:



2. Objectives

The main objectives of the proposed image recognition system are:

1. To develop a neural-network-based image classification system for Animals, Vehicles, and Buildings.
2. To prepare and preprocess the image dataset for neural-network training.
3. To design a Convolutional Neural Network (CNN) suitable for the classification task.
4. To train the CNN using labeled images and predict the category of unseen images.
5. To evaluate the model using accuracy, precision, recall, and F1-score.
6. To analyze classification errors using a confusion matrix.
7. To identify the strengths and limitations of the developed model.
8. To suggest possible improvements for better classification performance.

Expected Outcome

The system should accept an RGB image as input and automatically classify it as:

Animals , Vehicles , Buildings

3. Dataset Description

The image recognition system uses a selected subset of the CIFAR-100 dataset. Three categories were constructed to match the problem statement: Animals, Vehicles, and Buildings.

Three image classes were selected for each category:

Category	Selected Classes
Animals	Bear, Dolphin, Fox
Vehicles	Bicycle, Bus, Motorcycle
Buildings	Castle, House, Skyscraper

Each selected CIFAR-100 class contributes 500 training images and 100 testing images. Therefore, the final dataset contains an equal number of images in each category.

Category	Training Images	Testing Images
Animals	1,500	300
Vehicles	1,500	300
Buildings	1,500	300
Total	4,500	900

Dataset Characteristics

- Total training images: 4,500
- Total testing images: 900
- Number of categories: 3

- Training images per category: 1,500
- Testing images per category: 300
- Image format: RGB
- Image dimensions: $32 \times 32 \times 3$
- Task: Multi-class image classification

The dataset is balanced, since all three categories contain the same number of training and testing images.

Dataset Preparation

The selected CIFAR-100 classes were grouped into the three required categories. The resulting dataset was then used as input to the CNN model for training and testing.

4. Data Preprocessing

The selected images were prepared before being provided to the CNN model. Since the original CIFAR-100 images contain RGB pixel values ranging from 0 to 255, the pixel values were normalized to the range 0 to 1.

Preprocessing Steps

1. Image Selection:

Images belonging to the selected Animals, Vehicles, and Buildings classes were extracted.

2. Category Assignment:

The selected classes were mapped to three labels:

- $0 \rightarrow \text{Animals}$
- $1 \rightarrow \text{Vehicles}$
- $2 \rightarrow \text{Buildings}$

3. RGB Image Representation:

All images were retained as RGB images with dimensions $32 \times 32 \times 3$.

4. Pixel Normalization:

Each pixel value was divided by 255.

$$X_{\text{normalized}} = \frac{X}{255}$$

Thus, the pixel range changed from:

$$0-255 \rightarrow 0-1$$

5. Dataset Separation:

The final dataset contains 4,500 training images and 900 testing images, with equal representation of the three categories.

Preprocessing Output

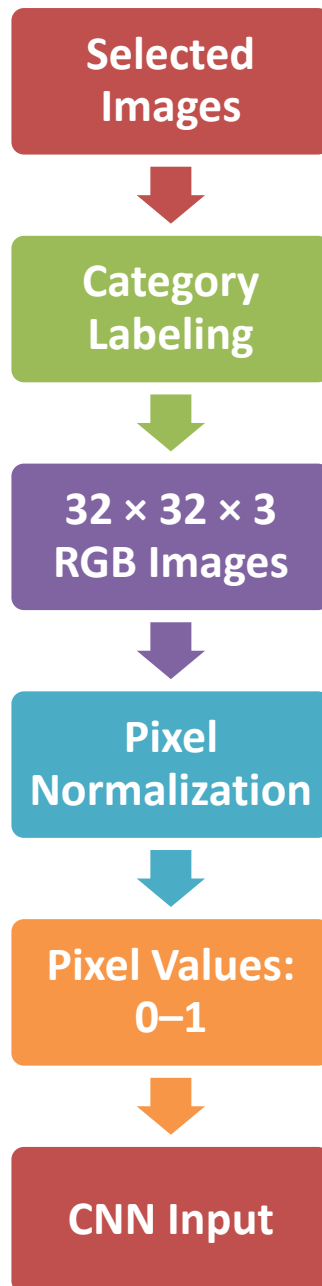
Training shape : (4500, 32, 32, 3)

Testing shape : (900, 32, 32, 3)

Minimum pixel value : 0.0

Maximum pixel value : 1.0

Preprocessing Flow



5. Selected AI Technique and Justification

Selected Technique: Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) was selected as the neural-network technique for the image recognition system. The task requires classifying RGB images into three categories: Animals, Vehicles, and Buildings.

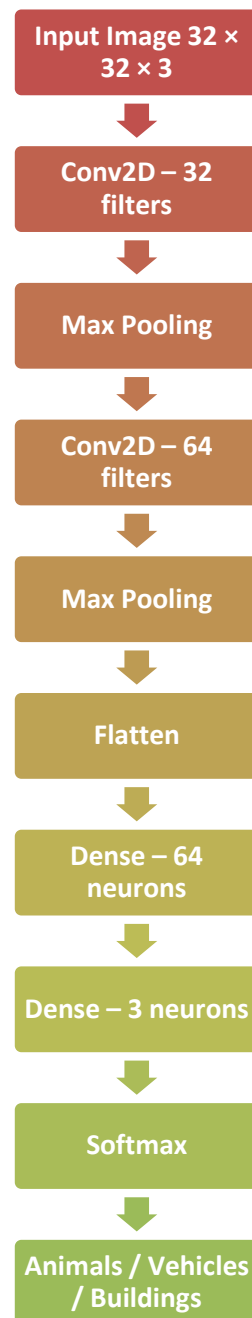
CNN is suitable because it can automatically learn visual patterns and features from image data through convolutional layers.

Why CNN is Suitable

1. **Feature Extraction:** Convolutional layers learn image features such as edges, shapes, and textures.
2. **Image-Specific Architecture:** CNNs are designed specifically for processing image data.
3. **Hierarchical Learning:** Initial layers learn simple features, while deeper layers learn more complex patterns.
4. **Multi-Class Classification:** The final softmax layer can assign an image to one of the three categories.
5. **Reduced Manual Feature Engineering:** Important image features are learned automatically during training.

Selected Architecture

The implemented CNN consists of:



The final output layer contains 3 neurons, one for each target category.

Justification

CNN was chosen because the problem is directly based on visual image patterns. Compared with a basic fully connected neural network, CNN preserves the spatial structure of images and performs feature extraction through convolution and pooling operations. Therefore, CNN is an appropriate neural-network technique for the proposed image classification system.

6. Model Design / Architecture

The proposed image recognition system uses a Convolutional Neural Network (CNN) to classify images into Animals, Vehicles, and Buildings. The model receives a $32 \times 32 \times 3$ RGB image and produces a probability for each of the three categories.

CNN Architecture

Layer	Configuration	Output Shape	Parameters
Input	$32 \times 32 \times 3$ RGB	$32 \times 32 \times 3$	—
Conv2D	32 filters, 3×3 , ReLU	$30 \times 30 \times 32$	896
Max Pooling	2×2	$15 \times 15 \times 32$	0
Conv2D	64 filters, 3×3 , ReLU	$13 \times 13 \times 64$	18,496
Max Pooling	2×2	$6 \times 6 \times 64$	0
Flatten	—	2,304	0
Dense	64 neurons, ReLU	64	147,520

Output	3 neurons, Softmax	3	195
Total			167,107

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_3 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten_1 (Flatten)	(None, 2304)	0
dense_2 (Dense)	(None, 64)	147,520
dense_3 (Dense)	(None, 3)	195

Total params: 167,107 (652.76 KB)
Trainable params: 167,107 (652.76 KB)
Non-trainable params: 0 (0.00 B)

Architecture Explanation

- **Convolutional layers** extract visual features from the input images.
- **ReLU activation** introduces non-linearity and helps the network learn complex patterns.
- **Max-pooling layers** reduce the spatial dimensions while retaining important features.
- **Flatten layer** converts the extracted feature maps into a one-dimensional vector.
- **Dense layer** performs high-level feature learning.
- **Softmax output layer** produces probabilities for the three target categories.

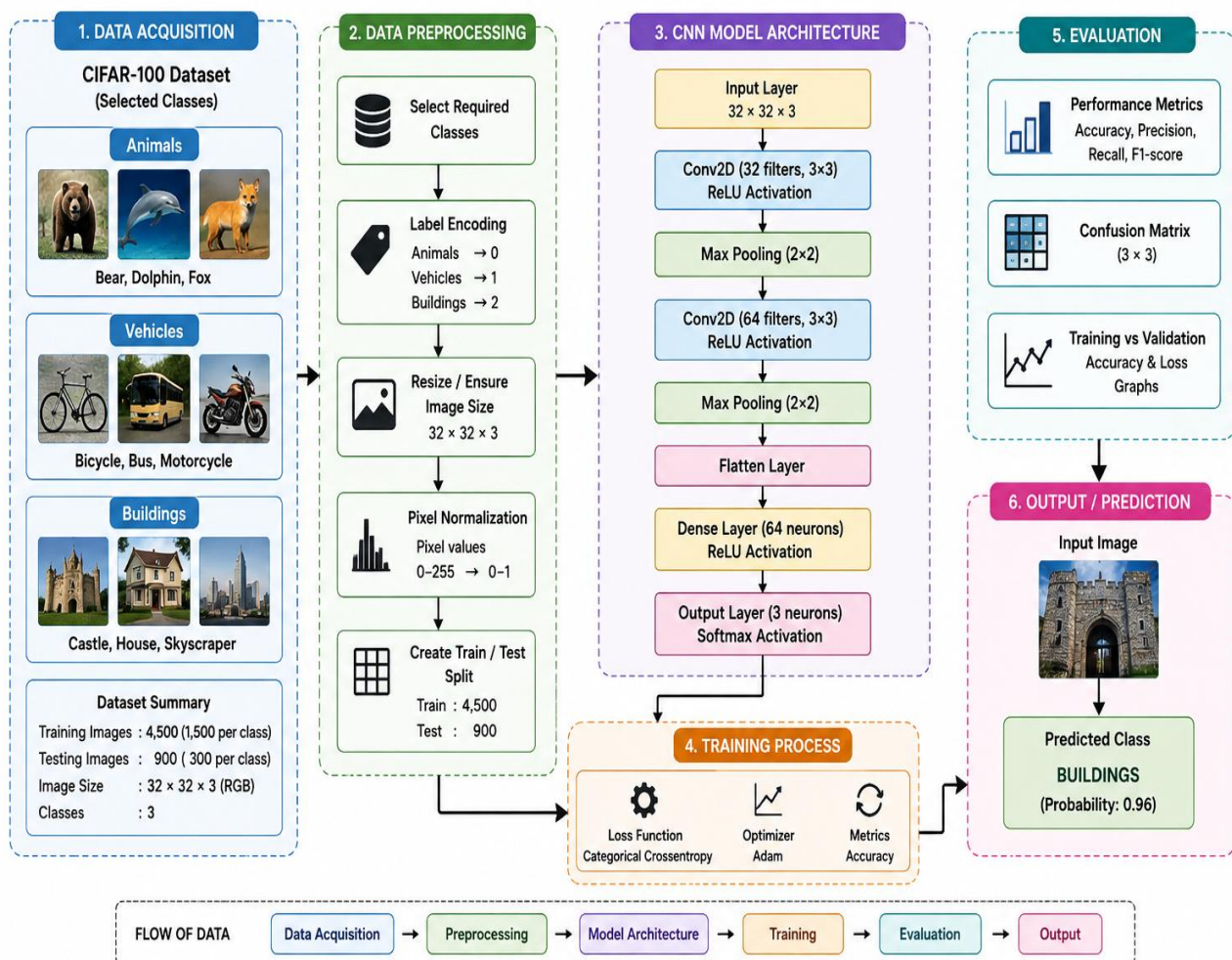
The three output neurons represent:

Output 0 → Animals

Output 1 → Vehicles

Output 2 → Buildings

Fig. 2 System architecture of the proposed CNN-based image recognition system.



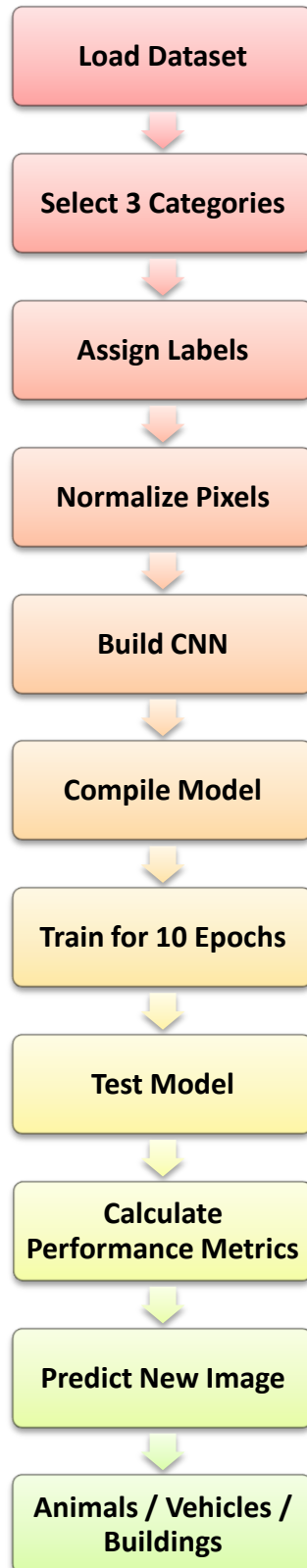
7. Algorithm / Pseudocode

Pseudocode

- START
- Load the selected image dataset.
- Select images belonging to:
 - Animals
 - Vehicles
 - Buildings
- Assign class labels:
 - Animals $\rightarrow 0$
 - Vehicles $\rightarrow 1$
 - Buildings $\rightarrow 2$
- Normalize pixel values: $\text{pixel} = \text{pixel} / 255$
- Divide the dataset into training and testing sets.
- Initialize the CNN model.
- Apply the first convolution and max-pooling layers.
- Apply the second convolution and max-pooling layers.
- Flatten the extracted features.
- Pass the features through the dense layer.
- Apply Softmax to the three output neurons.
- Compile the model using:

- Optimizer = Adam
- Loss = Sparse Categorical Crossentropy
- Metric = Accuracy
- Train the CNN for 10 epochs.
- Evaluate the trained model using the test dataset.
- Calculate:
 - Accuracy
 - Precision
 - Recall
 - F1-score
- Generate the confusion matrix.
- For a new image:
- Preprocess image
 - → Pass through CNN
 - → Calculate class probabilities
 - → Select class with highest probability
- Display the predicted category.
- END

Algorithm Flow



8. Implementation

The proposed image recognition system was implemented in Python using Google Colab. The TensorFlow/Keras framework was used to develop and train the CNN model, while NumPy, Matplotlib, and Scikit-learn were used for data processing and performance evaluation.

Implementation Environment

Component	Details
Programming Language	Python
Platform	Google Colab
Deep Learning Framework	TensorFlow / Keras
Data Processing	NumPy
Visualization	Matplotlib
Performance Evaluation	Scikit-learn
Dataset	Selected CIFAR-100 classes
Input Size	$32 \times 32 \times 3$
Output Classes	3

Implementation Steps

1. Dataset Loading

CIFAR-100 was loaded using TensorFlow's dataset API.

2. Class Selection

Three classes were selected for each required category:

- Animals: Bear, Dolphin, Fox
- Vehicles: Bicycle, Bus, Motorcycle
- Buildings: Castle, House, Skyscraper

3. Category Mapping

- Animals → 0
- Vehicles → 1
- Buildings → 2

4. Data Preparation

The selected images were combined to create a balanced dataset containing 4,500 training images and 900 testing images.

5. Normalization

Pixel values were converted from the range 0–255 to 0–1.

6. CNN Construction

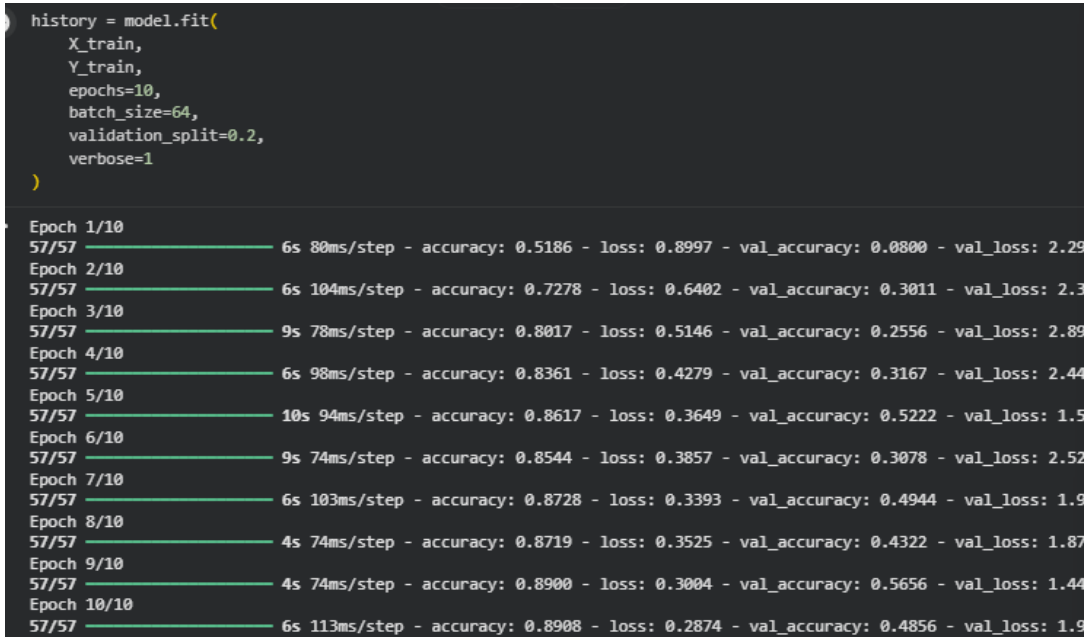
The CNN was implemented using two convolutional layers, two max-pooling layers, a flatten layer, a dense layer, and a three-neuron softmax output layer.

7. Model Compilation

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
print("Model compiled successfully.")  
  
Model compiled successfully.
```

8. Model Training

```
history = model.fit(  
    X_train,  
    Y_train,  
    epochs=10,  
    batch_size=64,  
    validation_split=0.2,  
    verbose=1  
)
```



Epoch	Step	Time/Step	Accuracy	Loss	Val Accuracy	Val Loss
1/10	57/57	6s 80ms/step	0.5186	0.8997	0.0800	2.29
2/10	57/57	6s 104ms/step	0.7278	0.6402	0.3011	2.3
3/10	57/57	9s 78ms/step	0.8017	0.5146	0.2556	2.89
4/10	57/57	6s 98ms/step	0.8361	0.4279	0.3167	2.44
5/10	57/57	10s 94ms/step	0.8617	0.3649	0.5222	1.5
6/10	57/57	9s 74ms/step	0.8544	0.3857	0.3078	2.52
7/10	57/57	6s 103ms/step	0.8728	0.3393	0.4944	1.9
8/10	57/57	4s 74ms/step	0.8719	0.3525	0.4322	1.87
9/10	57/57	4s 74ms/step	0.8900	0.3004	0.5656	1.44
10/10	57/57	6s 113ms/step	0.8908	0.2874	0.4856	1.9

9. Model Evaluation

The trained model was evaluated using the separate 900-image test dataset.

10. Performance Analysis

Classification report, confusion matrix, and training/validation graphs were generated to analyze the model.

Implementation Result

The trained CNN achieved a test accuracy of 78.11% on the unseen test dataset

9. Experimental Results

The trained CNN model was evaluated using the 900-image test dataset that was not used during model training. Training and validation performance were also recorded over 10 epochs.

9.1 Training Results

The model was trained using:

- **Epochs:** 10
- **Batch size:** 64
- **Optimizer:** Adam
- **Loss function:** Sparse Categorical Cross-Entropy
- **Validation split:** 20%

The training accuracy increased during training, reaching approximately 89% by the final epoch. Training loss decreased from approximately 0.87 to 0.29.

However, validation accuracy fluctuated and remained lower than training accuracy, indicating some degree of overfitting.

Fig. 3. Training and validation accuracy/loss over 10 epochs.



9.2 Test Results

The model was evaluated on the separate 900-image test dataset.

Metric	Result
Test Loss	0.7288
Test Accuracy	78.11%

The CNN correctly classified approximately 78% of the unseen test images.

```
Test Loss: 0.728783369064331
Test Accuracy: 0.781111211776733
Test Accuracy (%): 78.1111211776733
```

9.3 Classification Results

The classification report produced the following results:

Category	Precision	Recall	F1-Score
Animals	0.68	0.95	0.79
Vehicles	0.87	0.78	0.83

Buildings	0.86	0.61	0.71
Macro Average	0.80	0.78	0.78
Weighted Average	0.80	0.78	0.78

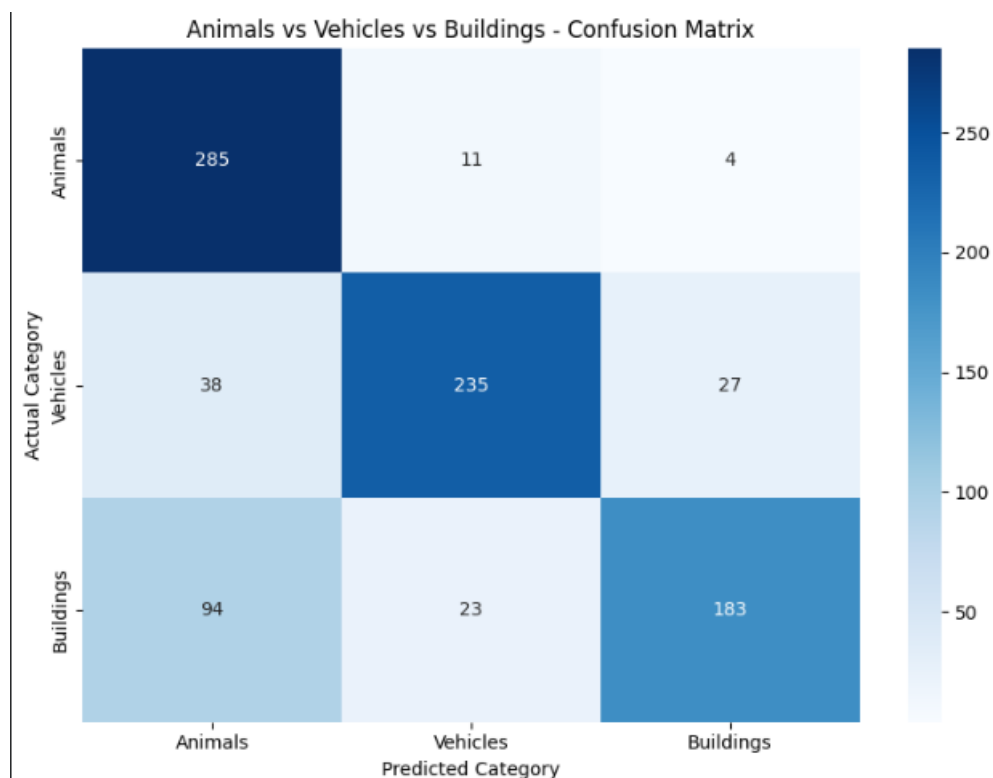
9.4 Confusion Matrix

The confusion matrix obtained from the 900 test images was:

Actual / Predicted	Animals	Vehicles	Buildings
Animals	285	11	4
Vehicles	38	235	27
Buildings	94	23	183

The diagonal values represent correctly classified images.

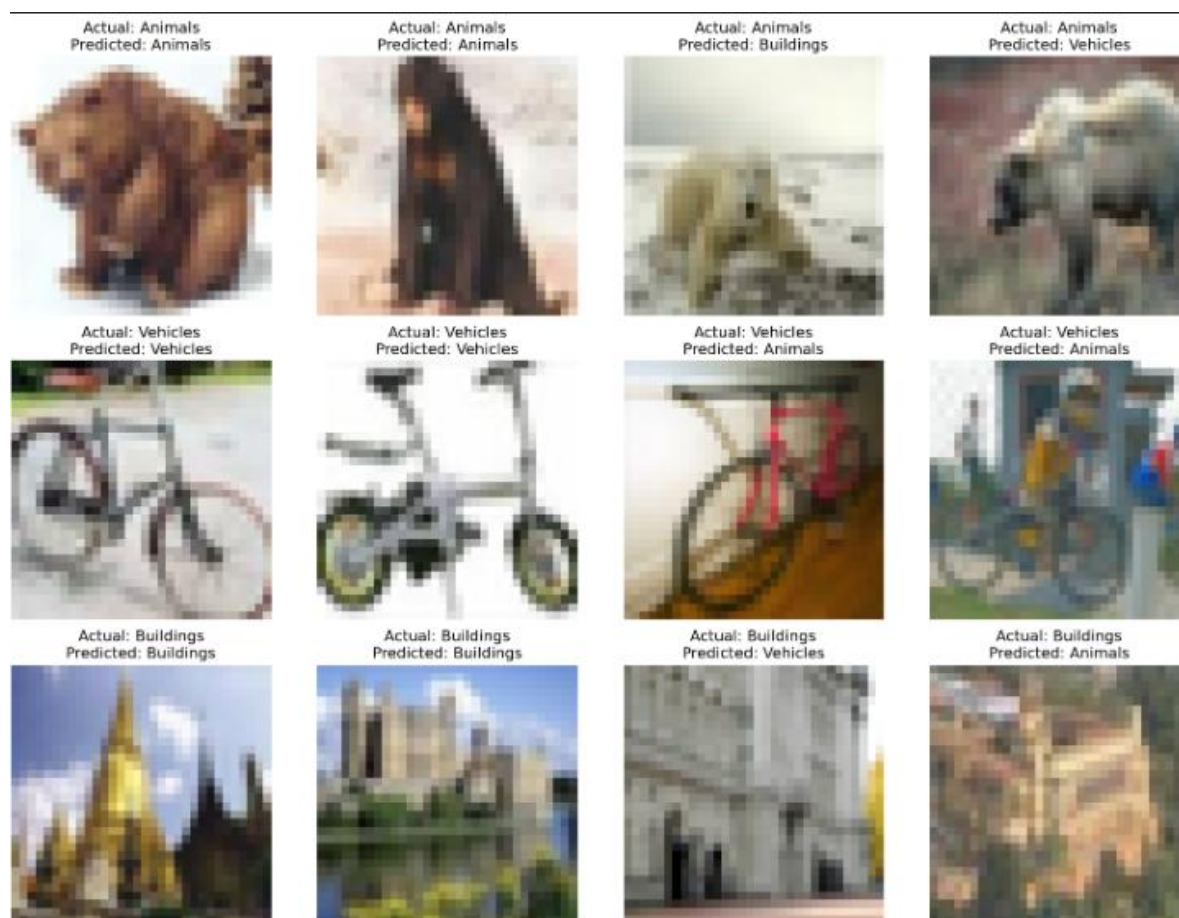
Fig. 4. Confusion matrix of the CNN-based image classification system.



9.5 Sample Predictions

The system was also tested using individual images from the test dataset. Both correctly and incorrectly classified examples were observed.

Fig. 5. Sample correct and incorrect predictions generated by the CNN model.



Main Experimental Finding

The proposed CNN achieved a 78.11% test accuracy, with a macro F1-score of 0.78 across the three categories. The results demonstrate that the model can distinguish the three broad image categories, although some confusion remains, particularly between Buildings and Animals.

10. Performance Evaluation

The CNN model was evaluated on the 900 unseen test images using accuracy, precision, recall, and F1-score.

10.1 Overall Performance

Metric	Value
Test Accuracy	78.11%
Macro Precision	0.80
Macro Recall	0.78
Macro F1-score	0.78
Test Loss	0.7288

10.2 Class-wise Performance

Category	Precision	Recall	F1-score
Animals	0.68	0.95	0.79
Vehicles	0.87	0.78	0.83
Buildings	0.86	0.61	0.71

The model achieved its highest F1-score for Vehicles (0.83). Animals achieved the highest recall (0.95), while Buildings had a lower recall of 0.61. The overall 78.11% test accuracy shows that the CNN can successfully classify the three target categories, although further improvement is required for building recognition.

11. Result Analysis

The CNN achieved a test accuracy of 78.11% with a macro F1-score of 0.78. The results show that the model can distinguish the three target categories, but its performance varies between classes.

- **Animals:** Achieved the highest recall (0.95), with 285 out of 300 animal images correctly classified.
- **Vehicles:** Achieved the highest F1-score (0.83), with 235 out of 300 images correctly classified.
- **Buildings:** Achieved 183 out of 300 correct classifications and the lowest recall (0.61).

The confusion matrix shows that the largest classification error occurred when 94 Building images were predicted as Animals. This explains the lower recall for the Buildings category.

The training graph also shows a noticeable difference between training and validation performance. Training accuracy increased to approximately 89%, while validation accuracy remained considerably lower and fluctuated. This indicates that the model experienced some overfitting.

Overall, the results demonstrate that the CNN learned useful visual patterns for the three-category classification task, but the model requires further improvement to distinguish Buildings from the other categories more reliably.

12. Limitations

The developed CNN-based image recognition system has the following limitations:

1. Low image resolution: The input images are only 32×32 pixels, which limits the amount of visual detail available to the model.
2. Limited categories: The system recognizes only three broad categories: Animals, Vehicles, and Buildings.
3. Classification errors: Some visually different classes are still confused, particularly Buildings being classified as Animals.
4. Overfitting: The training accuracy is considerably higher than the validation accuracy, indicating some overfitting.
5. Limited training data: The final model uses 4,500 training images, which limits the variety of images available for learning.
6. Dataset-specific performance: The model was trained using selected CIFAR-100 classes, so its performance may not directly

represent images from real-world datasets with different characteristics.

13. Future Enhancements

The proposed system can be improved in the following ways:

1. Use higher-resolution images to provide more visual details to the CNN.
2. Increase the training dataset with more diverse images from different environments and viewpoints.
3. Apply data augmentation such as rotation, cropping, flipping, and zooming to improve generalization.
4. Reduce overfitting using techniques such as dropout, regularization, and early stopping.
5. Improve the CNN architecture by adding more suitable convolutional layers or using transfer learning.
6. Expand the number of categories to recognize specific objects such as cars, buses, dogs, houses, and other individual classes.
7. Deploy the model as a real-time application where users can upload an image and receive its predicted category.

8. Use a larger and more diverse real-world dataset to improve performance outside the original training dataset.

14. Conclusion

A CNN-based image recognition system was developed to classify images into three categories: Animals, Vehicles, and Buildings. The selected dataset was prepared by grouping relevant CIFAR-100 classes and maintaining an equal number of images for each category.

The CNN was trained for 10 epochs using the Adam optimizer and achieved a test accuracy of 78.11% on 900 unseen test images. The model obtained a macro precision of 0.80, macro recall of 0.78, and macro F1-score of 0.78.

The results show that the model performed particularly well in recognizing Animals and Vehicles, while Buildings had lower recall because several building images were classified as animals. The training and validation curves also indicated some overfitting.

The system successfully demonstrates how a neural network can learn visual features and automatically classify images into predefined categories. Further improvements such as higher-resolution images, data augmentation, a larger dataset, and improved CNN architectures could increase its reliability and generalization.

15. References

- [1] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” University of Toronto, Toronto, Canada, 2009.
- [2] TensorFlow, “CIFAR-100 Dataset,” TensorFlow Documentation. [Online]. Available: [TensorFlow CIFAR-100 Documentation](#)
- [3] TensorFlow, “Convolutional Neural Network (CNN) Tutorial,” TensorFlow Documentation. [Online]. Available: [TensorFlow CNN Tutorial](#)
- [4] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

16. GitHub Repository Link

<https://github.com/kamallishree/MLA-0103/tree/main/CO4/AT1>

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	10%	9–10%–Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8%– Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	10%	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8%– Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	10%	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8%– Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4%–Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	10%	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8%– Appropriate technique selected with reasonable justification.	5–6%–Technique is acceptable but justification is limited.	0–4%–Inappropriate technique or no meaningful justification.

5. Model Design / Architecture	10%	9–10%– Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8%– Model is well designed with minor omissions.	5–6%– Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.
6. Algorithm/ Pseudocode	5%	5%– Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4%– Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1%– Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	15%	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12%– Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6%– Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	10%	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8%– Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4%– Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	10%	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8%– Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4%– Inappropriate metrics or performance evaluation is missing.

10. Result Analysis and Interpretation	5%	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4%–Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1%–Results are not meaningfully analyzed.
11.Limitations and Future Enhancements	5%	5% – Clearly identifies realistic limitations and propose technically relevant and feasible improvements.	4%–Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	10%	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				